

Documentação de Arquitetura Frontend

Projeto: Evolutto - Estrutura Angular e Gestão de Estado

Autor: Allan Victor Rodrigues Souza (Tech Lead)

Nota Arquitetural: O frontend adotará a abordagem de Standalone Components (Angular 14+), eliminando a necessidade de NgModules complexos. A gestão de estado reativo será feita utilizando Angular Signals para garantir alta performance nas atualizações de UI (XP, Moedas, Debuffs).

1. Estrutura de Diretórios e Componentes

A aplicação será dividida por domínios (features), seguindo a estrutura modular planejada no backend.

```

src/app/
├── core/                # Serviços Singleton, Interceptors e Guards
│   ├── auth/           # AuthService, AuthGuard, JwtInterceptor
│   └── store/          # UserStore (Signals para XP, Coins, Debuff)
├── shared/             # Componentes reutilizáveis (UI)
│   ├── ui/             # Botões, Modais, Cards, ProgressBars (XP)
│   └── pipes/          # Formataadores de moeda, tempo
├── features/           # Domínios da Aplicação (Lazy Loaded)
│   ├── habits/         # Domínio Core (Fase 1)
│   │   ├── components/ # HabitListComponent, HabitCardComponent
│   │   ├── services/   # HabitService (Chama os endpoints REST)
│   │   └── habits.routes.ts
│   ├── shop/           # Domínio da Lojinha (Fase 1)
│   │   ├── components/ # ShopBoardComponent, RewardCardComponent
│   │   ├── services/   # ShopService
│   │   └── shop.routes.ts
│   ├── social/         # Domínio de Grupos e Ranking (Fase 2)
│   │   ├── components/ # GroupDashboardComponent, LeaderboardComponent
│   │   └── services/   # GroupService
│   └── parental/       # Domínio Maker/Checker e Castigos (Fase 3)
│       ├── components/ # ParentDashboardComponent, PendingApprovalListComponent
│       └── services/   # ParentalService
└── app.routes.ts       # Roteamento principal

```

2. Roteamento e Lazy Loading

O `app.routes.ts` fará o carregamento sob demanda (Lazy Loading) das features para otimizar o tempo de carregamento inicial.

```
export const routes: Routes = [
  { path: 'login', component: LoginComponent },
  {
    path: 'habits',
    loadChildren: () => import('./features/habits/habits.routes').then(m =>
m.HABIT_ROUTES),
    canActivate: [AuthGuard]
  },
  {
    path: 'shop',
    loadChildren: () => import('./features/shop/shop.routes').then(m =>
m.SHOP_ROUTES),
    canActivate: [AuthGuard]
  },
  {
    path: 'parental',
    loadChildren: () => import('./features/parental/parental.routes').then(m =>
m.PARENTAL_ROUTES),
    canActivate: [AuthGuard, RoleGuard] // Apenas ROLE_PARENT acessa
  },
  { path: '', redirectTo: '/habits', pathMatch: 'full' }
];
```

3. Gestão de Estado (Angular Signals)

O controle do progresso do usuário (XP, Moedas, Nível e Penalidades) precisa refletir instantaneamente em toda a aplicação. Um `UserStoreService` utilizando Signals é a solução ideal.

```

import { Injectable, signal, computed } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class UserStoreService {
  // Estado reativo (Signals)
  readonly currentXp = signal<number>(0);
  readonly currentCoins = signal<number>(0);
  readonly currentLevel = signal<number>(1);
  readonly debuffCounter = signal<number>(0);

  // Computed Values (Derivados do estado atual)
  readonly isDebuffed = computed(() => this.debuffCounter() > 0);
  readonly xpToNextLevel = computed(() => this.currentLevel() * 1000 -
this.currentXp());

  // Ações para atualizar o estado
  updateProgress(newXp: number, newCoins: number, debuff: number) {
    this.currentXp.set(newXp);
    this.currentCoins.set(newCoins);
    this.debuffCounter.set(debuff);
  }

  spendCoins(cost: number) {
    this.currentCoins.update(coins => coins - cost);
  }
}

```

4. Integração Componente-Serviço (Exemplo de Hábito)

Fluxo de execução de um hábito no frontend chamando o backend estruturado.

```

@Component({
  selector: 'app-habit-card',
  standalone: true,
  template: `
    <div class="card">
      <h3>{{ habit.title }}</h3>
      <button (click)="execute()" [disabled]="isLoading">Concluir</button>
    </div>
  `,
})
export class HabitCardComponent {
  @Input() habit!: HabitResponse;
  isLoading = false;

  constructor(
    private habitService: HabitService,
    private userStore: UserStoreService
  ) {}

  execute() {
    this.isLoading = true;
    this.habitService.executeHabit(this.habit.id).subscribe({
      next: (res: ExecuteHabitResponse) => {
        // Atualiza a UI reativamente em toda a aplicação
        this.userStore.updateProgress(res.newTotalXp, res.newTotalCoins,
res.currentDebuffCounter);
        this.isLoading = false;
      },
      error: () => this.isLoading = false
    });
  }
}

```